The previous code creates a Movie object that wraps a record in the movies table, then accesses the record's title column by calling Movie#title and Movie#title=. But these methods are nowhere to be found in the source code. How can title and title= exist if they're not defined anywhere? You can find out by looking at how Active Record works.

The table name part is straightforward: Active Record looks at the name of the class through introspection and then applies some simple conventions. Since the class is named Movie, Active Record maps it to a table named movies. (This library knows how to find plurals for English words.)

What about methods such as title= and title, which access object attributes (*accessor methods* for short)? This is where metaprogramming comes in: Bob doesn't have to write those methods. Active Record defines them automatically, after inferring their names from the database schema. ActiveRecord::Base reads the schema at runtime, discovers that the movies table has two columns named title and director, and defines accessor methods for two attributes of the same name. This means that Active Record defines methods such as Movie#title and Movie#director= out of thin air while the program runs.

This is the "yang" to the introspection "yin": rather than just reading from the language constructs, you're writing into them. If you think this is an extremely powerful feature, you are right.

The "M" Word Again

Now you have a more formal definition of metaprogramming:

Metaprogramming is writing code that manipulates language constructs at runtime.

The authors of Active Record applied this concept. Instead of writing accessor methods for each class's attributes, they wrote code that defines those methods at runtime for *any* class that inherits from ActiveRecord::Base. This is what I meant when I talked about "writing code that writes code."

You might think that this is exotic, seldom-used stuff—but if you look at Ruby, as we're about to do, you'll see that it's used frequently.

Metaprogramming and Ruby

Remember our earlier talk about ghost towns and marketplaces? If you want to manipulate language constructs, those constructs must exist at runtime. In this respect, some languages are better than others. Take a quick glance at a few languages and how much control they give you at runtime.